

# How each regression algorithm works

*Theory, sklearn objects, and the exact notebook cells in notebooks/regression.ipynb that implement them.*

Problem · NYC yellow-cab trip duration (seconds)  
Notebook · notebooks/regression.ipynb, clusters A–J  
This file · documentation/regression.pdf

## THE TEN ALGORITHMS

**G1** Linear Regression

**G2** Ridge (L2)

**G3** Lasso (L1)

**G4** ElasticNet

**G5** Polynomial

**G6** Decision Tree

**G7** Random Forest

**G8** Gradient Boosting

**G9** SVR (RBF)

**G10** k-Nearest Neighbours

# Contents

---

- 1. Shared contract: one table, ten independent fits**
- 2. What `evaluate()` does after every algorithm**
- 3. G1 Linear Regression**
- 4. G2 Ridge Regression (L2)**
- 5. G3 Lasso Regression (L1)**
- 6. G4 ElasticNet**
- 7. G5 Polynomial Regression**
- 8. G6 Decision Tree**
- 9. G7 Random Forest**
- 10. G8 Gradient Boosting (histogram GBM)**
- 11. G9 Support Vector Regressor**
- 12. G10 k-Nearest Neighbours**
- 13. Mean baseline**
- 14. Cluster H: GridSearch, RandomizedSearch, top-two CV**
- 15. How to talk about a model in the viva**

*For each algorithm this booklet answers two questions: what is the model doing mathematically, and which lines in `regression.ipynb` actually run it. Code blocks are copied from the notebook, not invented.*

## 1. Shared contract: one table, ten independent fits

---

The notebook predicts `trip_duration` in seconds for NYC yellow-cab trips. It never uses `dropoff_datetime` (that column is the answer). It never uses `speed = distance / duration`. All ten official algorithms eat the same processed matrix from Cluster F and are scored on the same test trips.

### What Cluster F hands to every model

- `x_train_p / x_test_p` — numeric columns StandardScaled, vendor and store-and-fwd one-hot encoded. Scaler and encoder were fit on train only.
- `y_train / y_test` — duration in seconds. If the test split is larger than 50,000 rows, evaluation is capped at the same 50,000-row slice for everyone.
- Engineered columns: `haversine_km`, `manhattan_km`, `bearing`, `hour`, `dayofweek`, `month`, `is_weekend`, `is_rush_hour`, plus `passenger_count`.
- `random_state = 42` on the split, the trees, the subset draws, and the searches.

Linear, Ridge, Lasso, ElasticNet, Polynomial, Decision Tree, Random Forest, and HistGBM train on the full cleaned training table (about 279,000 rows). SVR trains on 12,000 rows; KNN on 40,000. The leaderboard column Train rows records that so the comparison stays honest.

## 2. What evaluate() does after every algorithm

Cluster Go defines one scorer. Every G-cell constructs its sklearn object, then calls this helper. The helper does not train the ten models in a loop — each algorithm still has its own cell, as the rubric asks.

```
model.fit(X_tr, y_tr)
y_pred = np.clip(model.predict(X_te), 1, None)
scores = five_fold_r2(model, X_tr, y_tr, name) # KFold(5, shuffle, seed 42)
# then print Test R2, RMSE, MAE, and 5-fold CV R2 mean ± std
```

### Step by step

- Fit on that algorithm's training rows (full table or the SVR/KNN subset).
- Predict the shared test set. Predictions below 1 second are clipped to 1 — a negative duration cannot happen. The clip is applied at predict time, not during training.
- Test metrics: R<sup>2</sup>, RMSE (seconds), MAE (seconds) via sklearn's `r2_score`, `root_mean_squared_error`, `mean_absolute_error`.
- 5-fold CV R<sup>2</sup> on the same training rows, using `clone(model)` so the already-fitted object is not reused. KFold shuffles with seed 42.
- The row is appended to leaderboard. The fitted object is stored in `fitted_models` for the Cluster I manual tester.

After G10 the notebook calls `show_all_metrics_table()`, which ranks every official model plus the mean baseline by test R<sup>2</sup> and displays Train rows, Test R<sup>2</sup>, RMSE, MAE, and 5-fold CV R<sup>2</sup> mean/std in one table. Cluster H1 prints that table again next to the bar chart.

*$R^2 = 1 - SS_{res} / SS_{tot}$ .  $RMSE = \sqrt{\text{mean}((y - \hat{y})^2)}$ , in seconds.  $MAE = \text{mean}(|y - \hat{y}|)$ , in seconds. 5-fold CV R<sup>2</sup> is the mean of five training-only holdouts; it is not computed on the test set.*

G1 · sklearn.linear\_model.LinearRegression

## Linear Regression

*Ordinary least squares. The straight-line baseline every other model has to beat.*

### How it works

Assume duration is a weighted sum of the processed features plus an intercept:

$$\hat{y} = w_o + w_1 x_1 + w_2 x_2 + \dots + w_p x_p$$

Training chooses the weights that minimise the residual sum of squares (ordinary least squares, OLS):

$$L(w) = \sum_i (y_i - w \cdot x_i)^2$$

When the columns are not perfectly collinear the unique minimiser is the normal equation  $w = (X^T X)^{-1} X^T y$ . sklearn solves this with a numerically stable SVD, not by inverting by hand. There is no learning rate. Default LinearRegression is a convex problem: you get the global minimum of that bowl.

Because Cluster F already scaled numerics, a coefficient of +80 on haversine\_km means “one standard deviation extra distance adds about 80 seconds”, not “one kilometre”. Haversine and Manhattan are collinear, so OLS may split one distance effect across two columns (one coefficient can even look negative). That is why Ridge exists.

### How the notebook implements it

```
lin = LinearRegression()
lin, y_pred_lin, _ = evaluate("Linear Regression", lin, X_train_p, y_train)

coef_lin = pd.Series(lin.coef_, index=feature_names)
                .sort_values(key=np.abs, ascending=False)
```

|                                   |                                                                                                                                |
|-----------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <b>Cluster</b>                    | G1, after the shared evaluate() helper in Go.                                                                                  |
| <b>Hyperparameters</b>            | None. sklearn default OLS.                                                                                                     |
| <b>Training rows</b>              | Full cleaned X_train_p.                                                                                                        |
| <b>After fit</b>                  | Prints the largest  coefficients  on scaled features, plus intercept. Distance should dominate; store_and_fwd_flag should not. |
| <b>Typical test R<sup>2</sup></b> | About 0.62 on this table — useful, not the winner.                                                                             |
| <b>Viva line</b>                  | “OLS fits a hyperplane by minimising squared error. It captures distance but not the congestion curve, so trees beat it.”      |

G2 · sklearn.linear\_model.Ridge

## Ridge Regression (L2)

*Same line as OLS, plus a penalty that shrinks large weights.*

### How it works

Start from OLS and add the squared length of the weight vector (the L2 penalty):

$$L(w) = \sum_i (y_i - w \cdot x_i)^2 + \alpha ||w||_2^2$$

$\alpha = 0$  is Linear Regression. As  $\alpha$  grows, weights are pulled toward zero but almost never hit exactly zero. The closed form becomes  $w = (X^T X + \alpha I)^{-1} X^T y$ . Adding  $\alpha I$  makes the matrix invertible even when two distance columns are nearly copies — that is the mathematical reason Ridge exists.

On this dataset haversine\_km and manhattan\_km both say “longer trip, longer duration”. OLS may give one a huge positive weight and the other a messy negative weight. Ridge splits the credit and keeps both modest. On ~279k rows OLS variance is already low, so test  $R^2$  often ties Linear. That is still a correct result: we implemented the regulariser, and mean |coefficient| is smaller than Linear’s.

### How the notebook implements it

```
ridge = Ridge(alpha=1.0, random_state=RANDOM_STATE)
ridge, _, _ = evaluate("Ridge Regression", ridge, X_train_p, y_train)
print(mean |lin.coef_| vs mean |ridge.coef_|)
```

Cluster H later runs GridSearchCV over  $\alpha$  in {0.001, 0.01, 0.1, 1, 10, 50, 100, 250, 1000}, scoring  $R^2$  with 5-fold CV, then refits Ridge(\*\*best\_params\_) on the full train and scores the same test set. If best  $\alpha$  is 1.0, G2 was already well specified.

|                        |                                                                                                                                         |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <b>Cluster</b>         | G2; tuned copy in H2 as “Ridge Regression (tuned)”.                                                                                     |
| <b>alpha</b>           | 1.0 in G2. Grid-searched in H2.                                                                                                         |
| <b>What to look at</b> | mean  coef_  vs Linear (Ridge smaller). Test $R^2$ almost tied with Linear.                                                             |
| <b>Viva line</b>       | “Ridge is OLS plus $\alpha$ times the square of the weights. It keeps every feature but shrinks them when two distances are collinear.” |

G3 · sklearn.linear\_model.Lasso

## Lasso Regression (L1)

*Same line, but the penalty can switch features off completely.*

### How it works

Replace Ridge's squared penalty with the sum of absolute weights (L1):

$$L(w) = \sum_i (y_i - w \cdot x_i)^2 + \alpha ||w||_1$$

$||w||_1 = |w_1| + |w_2| + \dots$ . The L1 diamond has corners on the axes, so the optimum often sits where some coordinates are exactly zero. That is automatic feature selection: Lasso can drop month or a vendor dummy if they do not pay rent.

There is no tidy closed form like Ridge. sklearn uses coordinate descent: freeze every weight except one, solve that one-dimensional problem, cycle, repeat. That is why the notebook sets `max_iter=20,000` — Lasso is iterative; Linear is not.

If no weights are zero, alpha is too small. If every weight is zero, alpha is too large. A few zeros with distance kept is the intended picture on this table.

### How the notebook implements it

```
lasso = Lasso(alpha=0.5, max_iter=20000, random_state=RANDOM_STATE)
lasso, _, _ = evaluate("Lasso Regression", lasso, X_train_p, y_train)
n_zero = (abs(lasso.coef_) < 1e-10).sum() # printed as a percentage
```

|                         |                                                                                                             |
|-------------------------|-------------------------------------------------------------------------------------------------------------|
| <b>Cluster</b>          | G3.                                                                                                         |
| <b>alpha / max_iter</b> | 0.5 and 20,000 iterations.                                                                                  |
| <b>After fit</b>        | Counts exact-zero coefficients and displays the non-zero ones sorted by  weight .                           |
| <b>Vs Ridge</b>         | Ridge shrinks; Lasso deletes. Neither is “more accurate” by definition.                                     |
| <b>Viva line</b>        | “Lasso uses an L1 penalty, so some weights become exactly zero. It is OLS with built-in feature selection.” |

G4 · sklearn.linear\_model.ElasticNet

## ElasticNet

*A mix of Lasso's sparsity and Ridge's stability.*

### How it works

Use both penalties at once:

$$L(w) = \text{sum} (y - w \cdot x)^2 + \text{alpha} [ l1\_ratio ||w||_1 + (1 - l1\_ratio) ||w||_2^2 ]$$

$l1\_ratio = 1$  is Lasso.  $l1\_ratio = 0$  is Ridge. The notebook uses 0.5: half of each. Lasso is greedy when two columns are twins — it may keep haversine and delete Manhattan even though both are valid. Ridge would keep both but never sparsify junk. ElasticNet can drop weak calendar bits and still share weight between the two distances. Training is coordinate descent, same family as Lasso.

### How the notebook implements it

```
enet = ElasticNet(alpha=0.2, l1_ratio=0.5,
                  max_iter=20000, random_state=RANDOM_STATE)
enet, _, _ = evaluate("ElasticNet Regression", enet, X_train_p, y_train)
```

|                                   |                                                                                                                       |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>Cluster</b>                    | G4.                                                                                                                   |
| <b>alpha, l1_ratio</b>            | 0.2 and 0.5. Slightly milder penalty than Lasso's 0.5 so it does not collapse to a constant.                          |
| <b>Typical test R<sup>2</sup></b> | Near the linear cluster, often a little below Ridge (the cost of extra sparsity).                                     |
| <b>Viva line</b>                  | "ElasticNet is alpha times a mix of L1 and L2. l1_ratio chooses how much feature-dropping versus shrinking you want." |

G5 · PolynomialFeatures + LinearRegression

## Polynomial Regression

*Still linear in the weights — nonlinear in the original columns.*

### How it works

Degree 1 is ordinary Linear Regression. Degree 2 builds extra columns: every square (hour<sup>2</sup>, haversine\_km<sup>2</sup>, ...) and every interaction (haversine\_km × hour, is\_rush\_hour × manhattan\_km, ...). Then OLS runs on that expanded matrix. The model is still  $\hat{y} = w \cdot z$ , but  $z$  contains products, so the surface in the original  $x$  is curved.

The interaction that matters here is distance × time-of-day. EDA showed the same kilometres cost more seconds at rush hour. A pure linear model can add a rush-hour offset; it cannot say "the distance slope itself gets steeper at 5pm". Degree 2 can. We do not try degree 3: the number of terms grows as  $O(p^3)$ , easy to overfit.

### How the notebook implements it

```
for degree in (1, 2):
    pipe = Pipeline([
        ("poly", PolynomialFeatures(degree=degree, include_bias=False)),
        ("lin", LinearRegression())
    ])
    pipe, y_pred, row = evaluate(
        f"Polynomial (degree {degree})", pipe, X_train_p, y_train,
        store=False)
# keep whichever degree has higher test R2 as "Polynomial Regression"
```

`include_bias=False` because `LinearRegression` already has an intercept. `store=False` so the two trial degrees do not both sit on the official leaderboard; only the winner is renamed Polynomial Regression and stored in

fitted\_models.

|                          |                                                                                                                                            |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Cluster</b>           | G5.                                                                                                                                        |
| <b>What usually wins</b> | Degree 2 (about 170 expanded columns vs 17). Test $R^2$ around 0.70 — between linear and trees.                                            |
| <b>Viva line</b>         | “We are not fitting a polynomial of duration. We expand features to degree 2, then fit a linear model so distance can interact with hour.” |

G6 · `sklearn.tree.DecisionTreeRegressor`

## Decision Tree

*Recursive binary splits. Not a line — a piecewise-constant map.*

### How it works

Start with every training trip in one box. Ask: which feature and which threshold cut this box into two children so durations inside each child are as tight as possible? Tight means low mean squared error. For a candidate split, predict the mean  $y$  of each child; impurity is the size-weighted MSE. Pick the split with the largest impurity drop. Repeat on each child.

A leaf predicts the mean duration of the trips that fell into it. The whole model is a set of axis-aligned rectangles, each with one constant  $\hat{y}$ . That is how a tree represents “if haversine\_km > 8 and hour in the rush band, predict ~1,400 s” without writing an interaction term by hand.

`max_depth` stops an unlimited tree from memorising train trips. `min_samples_leaf` stops one weird 3-hour ride from becoming its own rule. Feature importance is the total MSE reduction that feature was responsible for across all splits. Distance should rank first; hour and rush hour should appear.

### How the notebook implements it

```
tree = DecisionTreeRegressor(
    max_depth=12,
    min_samples_leaf=50,
    random_state=RANDOM_STATE,
)
tree, _, _ = evaluate("Decision Tree Regressor", tree, X_train_p, y_train)
# then bar-plot tree.feature_importances_ for the top 12 columns
```

|                                |                                                                                                                  |
|--------------------------------|------------------------------------------------------------------------------------------------------------------|
| <b>Cluster</b>                 | G6.                                                                                                              |
| <b>Why it beats Linear</b>     | It can isolate rush-hour long trips without a polynomial expansion.                                              |
| <b>Why a forest still wins</b> | One tree is high variance — a different 80/20 split grows a different tree.                                      |
| <b>Viva line</b>               | “A regression tree partitions feature space by MSE-reducing splits and predicts the mean duration in each leaf.” |

G7 · `sklearn.ensemble.RandomForestRegressor`

## Random Forest

*Many deep trees, each a little wrong, averaged.*

### How it works

A forest is bagging plus extra randomness:

- Bootstrap sample. Each tree is grown on  $n$  trips drawn with replacement. Some trips appear twice; others are left out (out-of-bag).
- Random feature subset at splits (when `max_features` is not 1.0). Stops every tree from making the same first split on haversine\_km.
- Average the trees. For a new trip, each tree votes a duration; the forest prediction is the mean of those votes.

One tree overfits its bootstrap quirks. Those quirks are weakly correlated across trees, so they cancel. Bias stays similar to a deep tree; variance falls. Feature importances are averaged impurity reductions — a ranking, not causal percentages.

### How the notebook implements it

```
forest = RandomForestRegressor(
    n_estimators=80, max_depth=18, min_samples_leaf=20,
    n_jobs=-1, verbose=1, random_state=RANDOM_STATE,
)
forest, _, _ = evaluate("Random Forest Regressor", forest, X_train_p, y_train)
```

Cluster H2 then `RandomizedSearchCV` on a 40,000-row subset (`TUNE_SAMPLE_SIZE`) with `n_iter=8`, `cv=3`, searching `n_estimators` in {60, 80, 120}, `max_depth` in {12, 16, 20, None}, `min_samples_leaf` in {10, 20, 40}, `max_features` in {sqrt, 0.5, 1.0}. The winning params are refit on the full training table and scored on the same test set as “Random Forest Regressor (tuned)”.

|                                      |                                                                                                                             |
|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <b>Cluster</b>                       | G7; tuned copy in H2.                                                                                                       |
| <b>Typical test <math>R^2</math></b> | About 0.79 — usually 2nd, just under HistGBM.                                                                               |
| <b>verbose=1</b>                     | So Run All is not a silent freeze while 80 trees grow.                                                                      |
| <b>Viva line</b>                     | “A random forest averages many deep trees grown on bootstrap samples, which cuts variance compared with one decision tree.” |

G8 · sklearn.ensemble.HistGradientBoostingRegressor

## Gradient Boosting (histogram GBM)

*Trees in a sequence, each one trained on the previous residual.*

### How it works

A forest builds trees in parallel and averages. Boosting builds them one after another. Start with  $F_0 = \text{mean}(y)$ . Residual of trip  $i$  is  $y_i - F_0$ . Grow a small tree  $h_1$  on those residuals. Update:

$$F_m(x) = F_{m-1}(x) + \text{learning\_rate} \cdot h_m(x)$$

A small learning\_rate (0.08) means each tree is a cautious step, so you need more trees (max\_iter=200) but you overfit less than one huge step. After  $M$  rounds the model is a sum of shallow trees. For squared-error loss the residual is exactly the negative gradient, so “fit the residual” is not a metaphor.

Why HistGradientBoostingRegressor, not GradientBoostingRegressor: the classical sklearn booster is too slow on ~279k rows. The histogram version bins each feature (max\_bins=255) and finds splits on bin edges. Same boosting idea, much faster, same family the rubric lists as Gradient Boosting. max\_depth=6 keeps each base tree a weak learner — boosting wants weak learners.

### How the notebook implements it

```
gbm = HistGradientBoostingRegressor(
    learning_rate=0.08, max_depth=6, max_iter=200,
    l2_regularization=0.1, verbose=1, random_state=RANDOM_STATE,
)
gbm, _, _ = evaluate("Gradient Boosting Regressor", gbm, X_train_p, y_train)
```

|                                   |                                                                                                                                                                                             |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Cluster</b>                    | G8. Leaderboard name is “Gradient Boosting Regressor” even though the class is HistGBM.                                                                                                     |
| <b>Typical test R<sup>2</sup></b> | About 0.80 — usually 1st on this table.                                                                                                                                                     |
| <b>Vs forest</b>                  | Forest reduces variance by averaging. Boosting reduces bias by sequentially fixing mistakes.                                                                                                |
| <b>Viva line</b>                  | “Gradient boosting adds trees that predict the leftover error of the current model, each scaled by a small learning rate. We use the histogram implementation so it finishes on 279k rows.” |

G9 · sklearn.svm.SVR + TransformedTargetRegressor

## Support Vector Regressor

*A tube around the data in a kernel-induced space. Not a tree, not OLS.*

### How it works

SVR looks for a function  $f(x) = w \cdot \phi(x) + b$  that is as flat as possible (small  $\|w\|$ ) while keeping every training residual inside an epsilon-insensitive tube. Errors smaller than epsilon cost nothing. Errors outside the tube pay a hinge cost, weighted by C:

$$\min \frac{1}{2} \|w\|^2 + C \sum_i x_i \text{ s.t. } |y_i - f(x_i)| \leq \epsilon + x_i$$

Large C fits training points tightly (risk overfitting). Small C is flatter and may underfit. Points on or outside the tube become support vectors; the others do not affect  $w$ .

$\phi(x)$  is never computed. The RBF kernel  $K(x, x') = \exp(-\gamma \|x - x'\|^2)$  is the inner product in an infinite-dimensional space. Two similar trips get a kernel value near 1; distant trips get  $\sim 0$ . The prediction is a weighted sum of kernels against the support vectors.

SVR's epsilon and C live in the scale of  $y$ . Raw duration is hundreds to thousands of seconds; default epsilon=0.1 is then meaningless and  $R^2$  collapses ( $\sim 0.2$ ). TransformedTargetRegressor(StandardScaler()) scales  $y$  to mean 0, std 1 for training, then inverts on predict. Features are already scaled in Cluster F.

Classic RBF-SVR is roughly  $O(n^2)$  to  $O(n^3)$ . 279k rows is not a laptop fit. We draw 12,000 training rows with seed 42, but still score the same 50,000 test rows as everyone else.

### How the notebook implements it

```
X_svr, y_svr = subset_train(SVR_TRAIN_SIZE)          # 12,000 rows
svr = TransformedTargetRegressor(
    regressor=SVR(kernel="rbf", C=1.0, epsilon=0.1, verbose=True),
    transformer=StandardScaler(),
)
svr, _, _ = evaluate("Support Vector Regressor", svr, X_svr, y_svr)
```

|                                      |                                                                                                                                                          |
|--------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Cluster</b>                       | G9.                                                                                                                                                      |
| <b>Typical test <math>R^2</math></b> | About 0.73 on 12k — strong for the sample size, not a fair fight with HistGBM.                                                                           |
| <b>Viva line</b>                     | “SVR fits a flat function in kernel space with an epsilon-tube. We scale the target, use RBF, and train on 12k rows because full-data SVR is $O(n^2)$ .” |

G10 · sklearn.neighbors.KNeighborsRegressor

## k-Nearest Neighbours

*No weights are learned. Prediction is a vote of nearby past trips.*

### How it works

KNN is a lazy learner. `fit()` stores the training matrix; it does not estimate  $w$ . To predict one new trip  $x$ , compute a distance to every stored trip, take the  $k$  closest, and average their durations. The notebook uses  $k=15$  and `weights='distance'`: closer neighbours count more (inverse-distance) instead of a flat mean.

Distance is Euclidean in the scaled feature space. That is why Cluster F's `StandardScaler` is mandatory here: without it, hour (0–23) would be crushed by a dummy 0/1, and kilometres would dominate everything. After scaling, one std of distance equals one std of hour in the metric.

`fit` prints ~0.0 s because it is a copy of the array. `predict` is  $O(n_{\text{train}} \times n_{\text{test}} \times n_{\text{features}})$ .  $279k \times 50k$  is not a viva-friendly wait, so we train on 40,000 rows and still score the same 50,000 test trips. KNN cannot extrapolate: a 30 km trip unlike anything in the 40k sample is averaged from the farthest “near” trips and will be too short.

### How the notebook implements it

```
X_knn, y_knn = subset_train(KNN_TRAIN_SIZE)          # 40,000 rows
knn = KNeighborsRegressor(
    n_neighbors=15, weights="distance", n_jobs=-1)
knn, _, _ = evaluate("K-Nearest Neighbors Regressor", knn, X_knn, y_knn)
show_all_metrics_table("Ten regression algorithms + mean baseline")
```

|                                      |                                                                                                                                                                                      |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Cluster</b>                       | G10. This cell also prints the full ranked metrics table for all ten models.                                                                                                         |
| <b>Typical test <math>R^2</math></b> | About 0.70 — third tier, ahead of linear, behind trees.                                                                                                                              |
| <b>Viva line</b>                     | “KNN predicts the distance-weighted average duration of the 15 closest training trips in scaled feature space. Fit is free; predict is the expensive part, so we train on 40k rows.” |

BASELINE · sklearn.dummy.DummyRegressor (not one of the ten)

## Mean predictor

*Always guess the training-set average duration.*

$\hat{y} = \text{mean}(y_{\text{train}})$  for every test trip. Test  $R^2$  is about 0 by construction. RMSE is the standard deviation of duration. Cluster G0 fits `DummyRegressor(strategy='mean')` and runs the same four mandatory metrics so a 0.62 linear  $R^2$  is visibly better than doing nothing. If a model ever lands near this row, it failed.

```
y_mean = np.full_like(y_test, fill_value=y_train.mean(), dtype=float)
dummy = DummyRegressor(strategy="mean")
dummy.fit(X_train_p, y_train)
# then the same  $R^2$  / RMSE / MAE / 5-fold CV  $R^2$  report as every official model
```

## 14. Cluster H: search, then reprint the two best

### GridSearchCV on Ridge

```
ridge_grid = GridSearchCV(
    Ridge(random_state=RANDOM_STATE),
    param_grid={"alpha": [0.001, 0.01, 0.1, 1, 10, 50, 100, 250, 1000]},
    scoring="r2", cv=5, n_jobs=-1, verbose=2,
)
ridge_grid.fit(X_train_p, y_train)
evaluate("Ridge Regression (tuned)",
        Ridge(**ridge_grid.best_params_, random_state=RANDOM_STATE),
        X_train_p, y_train)
```

A grid is an exhaustive product of the values you list. Inner 5-fold CV picks alpha; then evaluate() refits on all training rows and scores the held-out test set once. The search never sees test labels.

### RandomizedSearchCV on Random Forest

A forest has several knobs, so a full grid explodes. Randomized search draws 8 combinations on a 40k subset (cv=3) then refits the winner on the full train. n\_jobs=-1 and verbose=2 keep the laptop busy and the operator informed.

### Top-two 5-fold CV (rubric item)

Every G-cell already computed 5-fold CV  $R^2$ . Cluster H4 only selects the two official models with the highest test  $R^2$  (usually Gradient Boosting and Random Forest) and reprints their test  $R^2$  / RMSE / MAE / CV numbers. If the top two were SVR or KNN, those CV folds came from their 12k / 40k subsets — same rows used to fit.

## 15. How to talk about a model in the viva

If they name an algorithm, give the loss or rule first, then the notebook constructor, then one result on this taxi table.

| If they ask              | Start with                                                    | Then point at                        |
|--------------------------|---------------------------------------------------------------|--------------------------------------|
| How does Ridge work?     | OLS loss + $\alpha   w  ^2$                                   | G2 constructor; two distance columns |
| L1 vs L2?                | L2 shrinks, L1 can zero                                       | G3 zero-count printout               |
| Why polynomial?          | Degree-2 interactions                                         | G5 Pipeline; degree 2 usually wins   |
| How does a tree split?   | MSE impurity drop                                             | G6 importance bar chart              |
| Forest vs boosting?      | Average vs residual sequence                                  | G7 vs G8; HistGBM usually 1st        |
| Why subset SVR/KNN?      | $O(n^2)$ fit vs $O(n_{\text{train}} n_{\text{test}})$ predict | Train rows column on the table       |
| Did the scaler see test? | No. Cluster F fit on train only                               | ColumnTransformer in F               |
| Which is best?           | Highest test $R^2$ , confirmed by CV                          | G10 / H1 table, then H4 top two      |

*Closing sentence you can actually say: “We predicted NYC taxi duration from pickup context only, with no dropoff leak, on one encoded table, ten independent regressors, and the same test trips. Trees win because congestion is not linear; Ridge exists because the two distances are collinear; SVR and KNN are scored fairly but trained smaller because of complexity, not because they were ignored.”*

File: documentation/regression.pdf · Notebook: notebooks/regression.ipynb · Seed 42 throughout.